==========

APEX RETAIL GROUP | ANALYTICS TRAINING PROGRAM
DOCUMENT: CLASS NOTES (SESSION 5)
TOPIC: CLEANING IMPERFECT DATA — MISSING VALUES, TYPES & QUALITY
LEVEL: INTERMEDIATE (DAY 5)

==========

1. CONTINUITY & BUSINESS SCENARIO RECAP

In Session 4, you learned how to load raw tabular datasets, inspect DataFrame geometry (`df.shape`, `df.head()`), audit metadata (`df.info()`), and calculate basic summary statistics (`df.describe()`). During that discovery phase, you observed that real-world data often contains improperly formatted data types, missing metadata, and messy strings.

Now, in your fifth week at Apex Retail Group:

Rachel Torres, Customer Success Lead, submits an urgent ticket:

> **"Our customer returns database has become unreliable. We need to evaluate total refund costs and product return volumes for last month, but the export log is riddled with flaws.**
>
> **Some refund values are missing entirely, duplicate return records were logged by system glitches, product category labels have inconsistent casing and leading spaces (e.g., ' apparel ', 'Apparel', 'ELECTRONICS'), and return dates are stored as raw text.**
>
> **If we run analysis on this raw dataset, our numbers will be wrong. Can you clean and sanitize this customer returns dataset, document your data quality choices (imputation vs dropping), and output a reliable clean dataset with a health report?"**

In this session, you learn how real-world data quality choices directly impact business metrics. You will master handling missing data (`isnull()`, `fillna()`, `dropna()`), eliminating duplicate entries (`drop_duplicates()`), casting data types (`.astype()`), cleaning strings (`str.strip()`, `str.title()`), and parsing dates (`pd.to_datetime()`).

2. THE REALITY OF DATA IMPERFECTION & ANALYTICAL IMPACT

Real-world data is rarely pristine. Never blindly trust a dataset. Running summary statistics on uncleaned data leads to catastrophic business errors:

- **Missing Values** (`NaN`): If refund amounts are missing, computing `.sum()` ignores them, underreporting total company financial loss.
- **Inconsistent Text**: `'Apparel'` and `' apparel '` are treated as two separate product categories by Pandas, splitting category revenue in reports.
- **Duplicate Records**: System retries can create identical row entries, artificially inflating return volumes.
- **Incorrect Data Types**: Dates stored as strings prevent chronological sorting and period filtering.

3. PANDAS DATA CLEANING & TRANSFORMATION TOOLKIT

A. Detecting and Handling Missing Values (`NaN`)

- `df.isnull().sum()`: Counts missing values per column.
- `df.dropna(subset=['col'])`: Drops rows where critical identifier columns are missing.
- `df['col'].fillna(value)`: Imputes missing numeric values using calculated benchmarks (e.g., median or mean) or placeholder values (e.g., `"Unknown"`).

B. Removing Duplicate Records

- `df.duplicated().sum()`: Counts exact duplicate rows.
- `df.drop_duplicates(inplace=True)`: Permanently removes duplicate records.

C. Standardizing Text Strings

Pandas provides vectorised string methods via the `.str` accessor:

- `df['col'].str.strip()`: Removes leading and trailing whitespace.
- `df['col'].str.title()`: Capitalizes words consistently (e.g., `"ELECTRONICS"` -> `"Electronics"`).

D. Type Casting and Date Parsing

- `df['col'].astype(float)`: Converts numeric columns to floating-point numbers.
- `pd.to_datetime(df['col'])`: Converts text strings into explicit datetime objects, enabling date arithmetic and time extraction.

4. EXECUTING DATA CLEANING IN PANDAS

Below is the clean, annotated Python code required for Rachel's customer returns audit:

```python

import pandas as pd
import numpy as np

# Step 1: Load messy customer returns dataset
raw_returns = {
    "return_id": ["RET-101", "RET-102", "RET-102", "RET-103", "RET-104",
"RET-105", "RET-106"], # Duplicate RET-102
    "return_date": ["2026-09-01", "2026-09-02", "2026-09-02", "2026-09-03",
"2026-09-04", "2026-09-05", "2026-09-05"],
    "product_category": [" Electronics ", "Apparel", "Apparel", " HOME ",
"electronics", "Apparel", "Home"], # Dirty text
    "refund_amount": [150.00, 45.00, 45.00, np.nan, 210.00, np.nan, 85.00],
# Missing values
    "return_reason": ["Defective", "Wrong Size", "Wrong Size",
"Unsatisfactory", "Defective", "Wrong Size", "Defective"]
}

df = pd.DataFrame(raw_returns)

# Step 2: Identify initial quality defects
print("--- INITIAL MISSING VALUE COUNT ---")
print(df.isnull().sum())
print("\n--- INITIAL DUPLICATE COUNT ---", df.duplicated().sum())

# Step 3: Deduplicate exact duplicate rows
df = df.drop_duplicates().reset_index(drop=True)

# Step 4: Standardize string categories (strip whitespace and convert to
title case)
df["product_category"] = df["product_category"].str.strip().str.title()

# Step 5: Convert return_date string to explicit datetime data type
df["return_date"] = pd.to_datetime(df["return_date"])

# Step 6: Handle missing refund_amount values via median imputation
# We calculate median refund amount from valid entries to fill NaNs
median_refund = df["refund_amount"].median()
df["refund_amount"] = df["refund_amount"].fillna(median_refund)
```

```
# Step 7: Verify cleaned DataFrame state and data types
print("\n--- CLEANED DATAFRAME ---")
print(df)
print("\n--- CLEANED DATA TYPES ---")
print(df.dtypes)
```

INTERPRETING NUMBERS & FORMULATING BUSINESS RECOMMENDATIONS

A. Observation (The Data Quality Audit):

- Deduplication: Reduced row count from 7 to 6 by removing 1 exact duplicate entry (`RET-102`).
- Category Standardization: Consolidated chaotic categories (`" Electronics "`, `"electronics"`) into 3 clean, uniform categories (`Electronics`, `Apparel`, `Home`).
- Missing Value Imputation: Imputed 2 missing `refund_amount` entries using the category median ($117.50), bringing total audited refund value to $652.50.
- Date Formatting: Cast `return_date` from `object` (string) to `datetime64[ns]`.

B. Interpretation & Business Meaning:

- Uncleaned data would have miscounted return volumes due to duplicate system logs.
- Category revenue loss was previously split across inconsistent string names, making `Electronics` appear smaller than it actually was.
- Imputing missing refund amounts using the median allows Finance to account for estimated liabilities without throwing away valid return logs.

C. Professional Recommendation to Rachel Torres (Customer Success Lead):

"The customer returns dataset has been cleaned, deduplicated, and standardized. System duplicates were removed, resolving artificial volume inflation by 14.3%. Category formatting was unified across Electronics, Apparel, and Home, and missing refund figures were imputed using the $117.50 median return baseline.

Recommendation: Approve this sanitized dataset for operational return analysis. Require IT to enforce input validation on the web return form to prevent leading whitespace and unformatted category strings at entry."

CORE MINDSET HABITS REINFORCED TODAY

1. Transparent Imputation: Never silently fill or drop missing values. Document every data transformation and explain why imputation choices (e.g., median vs mean) preserve financial accuracy.
2. Standardize Strings Early: Clean whitespace and letter casing before performing categorical grouping or filtering.